Binary Search & Strings

⇒ What is Binary Search?

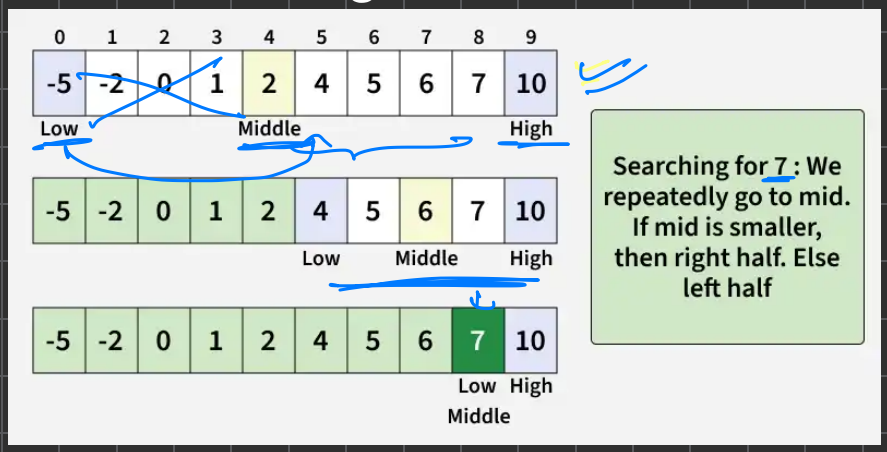

Image from GeeksForGeeks

```
n = len(arr)
low = 0, high = n-1
while (low <= high)
{
    mid = (low+high)//2
    if (arr(mid) == target)
        return mid
    else if (target > arr(mid))
        low = mid+1
    else
        high = mid-1
}
```

⇒ How to determine we need to apply Binary Search?

⇒ Applying two-pointers & sliding window on strings.

T.C = $O(\log(n))$

S.C = $O(1)$

PROBLEM - 1

34. Find First and Last Position of Element in Sorted Array

Medium | Topics | Companies

Given an array of integers `nums` sorted in non-decreasing order, find the starting and ending position of a given `target` value.

If `target` is not found in the array, return `[-1, -1]`.

~~You must write an algorithm with O(log n) runtime complexity.~~

Example 1:

```
Input: nums = [5,7,7,8,8,10], target = 8
Output: [3,4]
```

Example 2:

```
Input: nums = [5,7,7,8,8,10], target = 6
Output: [-1,-1]
```

Example 3:

```
Input: nums = [], target = 0
Output: [-1,-1]
```

Constraints:

- `0 <= nums.length <= 10^5`
- `-10^9 <= nums[i] <= 10^9`
- `nums` is a non-decreasing array.
- `-10^9 <= target <= 10^9`

Brute force:

nums = [5, 7, 7, 8, 8, 10], target = 8

index = 0 1 2 3 4 5

first = -1, last = -1

T.C = O(n) ; n ⇒ no. of ele.

S.C = O(1)

first occurance of target

nums = [5, 7, 7, 8, 8, 10], target = 8

0 1 2 3 4 5

l, m, h, m

n = 6

first = 3

last occurance of target

nums = [5, 7, 7, 8, 8, 10], target = 8

0 1 2 3 4 5

m, h, l

last = 4

while (low <= high)

```
find_first_occur{
    l=0, h=n-1, first=-1
    while (l<=h)
    {
        m= (l+h)//2
        if (nums[m] == target)
        {   first = m
            h = m-1
        }
```

```
find_last_occur{
    l=0, h=n-1, las=-1
    while( l<=h)
    {   m=(l+h)//2
        if (nums[m]==target)
        {   last = m
            l=m+1
        }
```

```
else if (target > nums[m])
        l = m+1
else
        h = m-1
}
return first
}
```

```
else if (target > nums[m])
        l=m+1
else
        h=m-1
}
return last
```

PROBLEM-2

3. Longest Substring Without Repeating Characters

Medium | Topics | Companies | Hint

Given a string `s`, find the length of the **longest** substring without duplicate characters.

Example 1:

```
Input: s = "abcabcbb"
Output: 3
Explanation: The answer is "abc", with the length of 3. Note that "bca" and "cab" are also correct answers.
```

Example 2:

```
Input: s = "bbbbb"
Output: 1
Explanation: The answer is "b", with the length of 1.
```

Example 3:

```
Input: s = "pwwkew"
Output: 3
Explanation: The answer is "wke", with the length of 3.
Notice that the answer must be a substring, "pwke" is a subsequence and not a substring.
```

Constraints:

- `0 <= s.length <= 5 * 10^4`
- `s` consists of English letters, digits, symbols and spaces.

s = "abcabcbb"

a
ab
abc
abcax
abcabx

b
bc
bca
bcabx

hash

a	b	c	d ...
~~0~~	~~0~~	~~0~~	0
~~1~~	1	1	

2

max_len = ~~2~~ 3

Brute force:

```
max_len = 0
for i: 0 -> n {
    hash[256] = {0}
    for j: i -> n
    {
        if (hash[s[j]] > 1) break;
        max_len = max(max_len, j-i+1)
        hash[s[j]] += 1
    }
}
```

(j - i + 1)

i: 0 1 2 3 4
(a, b, c, a b)
i j

a
ab
abc

a	b	c	d
~~0~~	~~0~~	~~0~~	0
1	1	1	

a	b	c	d	e	f	...	...
[0	0	0	0	0	0	0	0]

T.C = $O(n^2) + O(\log(256))$

S.C = $O(256)$

	0		2	3	4	5	6	7 ↓ L
s =	a	b	c	a	b	c	b	b
	l, r							↑ r

$\cancel{r} = r$

len = r − l + 1

max_len = ~~x~~ ~~x~~ ~~x~~ 3

(c, 5)
(b, 7)
(a, 3)

PROBLEM - 3

33. Search in Rotated Sorted Array

Medium | Topics | Companies

There is an integer array `nums` sorted in ascending order (with **distinct** values).

Prior to being passed to your function, `nums` is **possibly left rotated** at an unknown index `k` (`1 <= k < nums.length`) such that the resulting array is `[nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]]` (**0-indexed**). For example, `[0,1,2,4,5,6,7]` might be left rotated by `3` indices and become `[4,5,6,7,0,1,2]`.

Given the array `nums` **after** the possible rotation and an integer `target`, return *the index of* `target` *if it is in* `nums`, *or* `-1` *if it is not in* `nums`.

You must write an algorithm with `O(log n)` runtime complexity.

Example 1:

```
Input: nums = [4,5,6,7,0,1,2], target = 0
Output: 4
```

Example 2:

```
Input: nums = [4,5,6,7,0,1,2], target = 3
Output: -1
```

Example 3:

```
Input: nums = [1], target = 0
Output: -1
```

target = 0

nums = [4, 5, 6, 7, 0, 1, 2]
0 1 2 3 4 5 6

Brute force: linear search
T.C = O(n)
S.C = O(1)

m, target = 0

nums = [4, 5, 6, 7, 0, 1, 2]
0 1 2 3 4 5 6

m = (l+h)//2 ; if (nums[mid] == target) set mid

[4, 5, 6, 7, 0]
0 1 2 3 4
l

target = 0

[~~3, 4, 5~~, 0, 1, 2]

if nums[low] <= nums[mid]:
  if (nums[low] <= target < nums[mid])
    high = mid - 1
  else
    low = mid + 1
else
  if (nums[mid] < target <= nums[high])
    low = mid + 1
  else
    high = mid - 1

return -1

[~~3, 4, 5, 6~~, 7, 0, 1, 2]
~~l~~ ~~m~~ l m h

, target = 0

m = (l+h)//2